Modern Data Engineering Roadmap

8-Week Engineering Sprint | 12-15 hrs/week

For engineers with SQL + BI experience
transitioning to modern data engineering.

Target Stack

Python | PostgreSQL | Docker | Airflow | dbt
Spark | Kafka | AWS (S3 + Redshift) | Terraform
GitHub Actions | Great Expectations | Delta Lake

Weekly Time Split (15 hrs)

Learning: 5 hrs | Hands-on: 7 hrs | Reading: 3 hrs

What's Inside This Guide

- 8 week overviews with topics, resources, goals & deliverables
- 40+ working code examples (Python, SQL, YAML, HCL, Bash)
- Architecture diagrams for every major component
- 10 tool cheat sheets in the Cookbook appendix

WEEK 1

Python for Data Engineering (Production Style)

Summary

This week focuses on leveling up your Python beyond scripting into production-grade engineering. You already know Python basics— now you'll learn how professional data engineers structure projects: proper packaging with pyproject.toml, isolated virtual environments, structured logging (not print statements), and building APIs that ingest data from external sources. The emphasis is on writing code that is testable, maintainable, and deployable— not just functional.

Key Topics

- Project packaging: pyproject.toml, src layout, entry points
- Virtual environments: venv, pip-tools, dependency pinning
- Structured logging: Python logging module, log levels, JSON formatters
- HTTP clients: requests/httpx for API ingestion with retry logic
- Error handling: custom exceptions, graceful degradation
- Type hints and dataclasses for data contracts
- FastAPI basics: building lightweight ingestion endpoints
- Config management: environment variables, .env files, pydantic-settings

Resources

- **Python Official Tutorial** https://docs.python.org/3/tutorial/
- **Real Python (production tutorials)** https://realpython.com/
- **FastAPI Documentation** https://fastapi.tiangolo.com/
- **Kaggle Datasets** https://www.kaggle.com/datasets
- **NYC Open Data** https://opendata.cityofnewyork.us/

GOAL: Build production-quality Python skills— learn to write code that other engineers can read, test, and deploy with confidence.

DELIVERABLES

- ☑ Python project with proper packaging (pyproject.toml, src layout)
- ☑ FastAPI ingestion service that pulls from a public API (e.g., NYC Open Data)
- ☑ Raw JSON saved to local storage with timestamped filenames
- ☑ Structured logging throughout (no print statements)
- ☑ Unit tests with pytest covering the ingestion logic

Week 1 Deep-Dive: Python Code Examples

Project Structure

```
taxi-ingestion/
+-- pyproject.toml          # project metadata + deps
+-- src/
|   +-- taxi_ingestion/
|       +-- __init__.py
|       +-- app.py          # FastAPI application
|       +-- config.py       # pydantic-settings
|       +-- logging_config.py
|       +-- ingestion.py    # download logic
|       +-- models.py       # data contracts
+-- tests/
|   +-- test_app.py
|   +-- test_ingestion.py
|   +-- conftest.py
+-- data/
|   +-- raw/                # downloaded files land here
+-- .env                    # local overrides
+-- Dockerfile
```

pyproject.toml — Modern Python project configuration

pyproject.toml

```
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[project]
name = "taxi-ingestion"
version = "0.1.0"
requires-python = ">=3.11"
dependencies = [
    "httpx>=0.27",
    "fastapi>=0.110",
    "uvicorn[standard]>=0.29",
    "pydantic-settings>=2.2",
]

[project.optional-dependencies]
dev = ["pytest>=8.0", "ruff>=0.4", "pytest-cov"]

[project.scripts]
taxi-ingest = "taxi_ingestion.app:main"

[tool.ruff]
line-length = 100
target-version = "py311"
```

Structured Logging — Replace print() with proper logging

Python — logging_config.py

```python
import logging
import json
import sys

class JSONFormatter(logging.Formatter):
    def format(self, record):
        return json.dumps({
            "ts": self.formatTime(record),
            "level": record.levelname,
            "module": record.module,
            "msg": record.getMessage(),
        })

def get_logger(name: str) -> logging.Logger:
    logger = logging.getLogger(name)
    handler = logging.StreamHandler(sys.stdout)
    handler.setFormatter(JSONFormatter())
    logger.addHandler(handler)
    logger.setLevel(logging.INFO)
    return logger

log = get_logger("ingestion")
log.info("Pipeline started", extra={"run_id": "abc123"})
```

FastAPI Ingestion Endpoint — HTTP API for triggering data pulls

Python — app.py

```python
from fastapi import FastAPI, BackgroundTasks, HTTPException
from pydantic import BaseModel, field_validator
import httpx
from pathlib import Path
from datetime import datetime

app = FastAPI(title="Taxi Data Ingestion")

class IngestRequest(BaseModel):
    year: int
    month: int

    @field_validator("month")
    @classmethod
    def valid_month(cls, v):
        if not 1 <= v <= 12:
            raise ValueError("month must be 1-12")
        return v

async def download_parquet(year: int, month: int):
    url = (
        f"https://d37ci6vzurychx.cloudfront.net/trip-data/"
        f"yellow_tripdata_{year}-{month:02d}.parquet"
    )
    out = Path(f"data/raw/{year}_{month:02d}.parquet")
    out.parent.mkdir(parents=True, exist_ok=True)
    async with httpx.AsyncClient(timeout=120) as c:
        resp = await c.get(url)
        resp.raise_for_status()
        out.write_bytes(resp.content)

@app.post("/ingest")
async def ingest(req: IngestRequest, bg: BackgroundTasks):
    bg.add_task(download_parquet, req.year, req.month)
    return {"status": "accepted", "period": f"{req.year}-{req.month:02d}"}

@app.get("/health")
async def health():
    return {"status": "ok"}
```

Data Contracts with Dataclasses — Typed records for pipeline data

Python — models.py

```python
from dataclasses import dataclass, asdict
from datetime import datetime
from typing import Optional

@dataclass(frozen=True)
class TaxiTrip:
    pickup_datetime: datetime
    dropoff_datetime: datetime
    passenger_count: int
    trip_distance: float
    fare_amount: float
    tip_amount: float
    total_amount: float
    payment_type: Optional[str] = None

    def to_dict(self) -> dict:
        return asdict(self)

    @property
    def duration_minutes(self) -> float:
        delta = self.dropoff_datetime - self.pickup_datetime
        return delta.total_seconds() / 60
```

httpx Ingestion with Retry — Resilient HTTP downloads

Python — ingestion.py

```python
import httpx
from tenacity import retry, stop_after_attempt, wait_exponential
from pathlib import Path
from logging_config import get_logger

log = get_logger("ingestion")

@retry(
    stop=stop_after_attempt(3),
    wait=wait_exponential(multiplier=1, min=2, max=30),
    reraise=True,
)
def download_file(url: str, dest: Path) -> Path:
    log.info(f"Downloading {url}")
    dest.parent.mkdir(parents=True, exist_ok=True)
    with httpx.Client(timeout=120, follow_redirects=True) as c:
        with c.stream("GET", url) as resp:
            resp.raise_for_status()
            with open(dest, "wb") as f:
                for chunk in resp.iter_bytes(chunk_size=8192):
                    f.write(chunk)
    log.info(f"Saved {dest} ({dest.stat().st_size:,} bytes)")
    return dest
```

Pytest Tests — Testing the ingestion logic

Python — test_app.py

```python
import pytest
from httpx import AsyncClient
from unittest.mock import patch, AsyncMock
from app import app

@pytest.mark.anyio
async def test_ingest_returns_accepted():
    async with AsyncClient(app=app, base_url="http://test") as c:
        resp = await c.post("/ingest",
            json={"year": 2024, "month": 1})
        assert resp.status_code == 200
        data = resp.json()
        assert data["status"] == "accepted"
        assert data["period"] == "2024-01"

@pytest.mark.anyio
async def test_ingest_validates_month():
    async with AsyncClient(app=app, base_url="http://test") as c:
        resp = await c.post("/ingest",
            json={"year": 2024, "month": 13})
        assert resp.status_code == 422

@pytest.mark.anyio
async def test_health():
    async with AsyncClient(app=app, base_url="http://test") as c:
        resp = await c.get("/health")
        assert resp.json()["status"] == "ok"
```

Config with pydantic-settings — Environment-driven configuration

Python — config.py

```python
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    api_base_url: str = "https://d37ci6vzurychx.cloudfront.net"
    data_dir: str = "data/raw"
    log_level: str = "INFO"
    max_retries: int = 3

    model_config = {"env_prefix": "TAXI_"}

settings = Settings()  # reads TAXI_API_BASE_URL, TAXI_DATA_DIR, etc.
```

WEEK 2

Modern SQL + Data Modeling

Summary

SQL is the lingua franca of data engineering, and this week you'll move well beyond SELECT statements. You'll master window functions (ROW_NUMBER, LAG, LEAD, NTILE), CTEs for readable query composition, and query optimization using EXPLAIN ANALYZE. The second half focuses on dimensional modeling— the Kimball methodology that underpins every modern data warehouse. You'll learn to identify facts (measurable events) vs. dimensions (descriptive context), design star schemas, and handle slowly changing dimensions (SCDs). This is the foundation that makes dbt, Redshift, and Snowflake work effectively.

Key Topics

- Window functions: ROW_NUMBER, RANK, LAG, LEAD, NTILE, running totals
- Common Table Expressions (CTEs) and recursive queries
- Query optimization: EXPLAIN ANALYZE, index strategies, partitioning
- Dimensional modeling: facts vs. dimensions, grain definition
- Star schema design: fact tables, dimension tables, surrogate keys
- Slowly Changing Dimensions (SCD Type 1, 2, 3)
- Data normalization vs. denormalization trade-offs
- PostgreSQL-specific features: JSONB, array types, materialized views

Resources

- **Mode SQL Tutorial** https://mode.com/sql-tutorial/
- **PostgreSQL Documentation** https://www.postgresql.org/docs/
- **Kimball Group Resources** https://www.kimballgroup.com/
- **Chinook Sample Database** https://github.com/lerocha/chinook-database

GOAL: Master analytical SQL patterns and design a dimensional model from scratch— the skill that separates data engineers from Python scripters.

DELIVERABLES

- ☑ Chinook DB loaded into PostgreSQL with analytical query workbook
- ☑ 10+ analytical queries using window functions, CTEs, and aggregations
- ☑ Star schema design document (ERD) for a business domain of your choice
- ☑ Fact + Dimension DDL scripts with surrogate keys and indexes
- ☑ Query performance comparison: before/after indexing with EXPLAIN ANALYZE

Week 2 Deep-Dive: SQL Code Examples

Window Functions — Analytics beyond GROUP BY

SQL — Window functions

```sql
-- Rank trips by fare within each day
SELECT
    pickup_date,
    trip_id,
    fare_amount,
    ROW_NUMBER() OVER (
        PARTITION BY pickup_date
        ORDER BY fare_amount DESC
    ) AS fare_rank,
    LAG(fare_amount) OVER (
        PARTITION BY pickup_date
        ORDER BY pickup_datetime
    ) AS prev_fare,
    SUM(fare_amount) OVER (
        PARTITION BY pickup_date
        ORDER BY pickup_datetime
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS running_total
FROM trips
WHERE pickup_date = '2024-01-15';
```

CTEs for Readable Queries — Break complex logic into named steps

SQL — CTEs

```sql
WITH daily_stats AS (
    SELECT
        pickup_date,
        COUNT(*) AS trip_count,
        AVG(fare_amount) AS avg_fare,
        PERCENTILE_CONT(0.5) WITHIN GROUP
            (ORDER BY fare_amount) AS median_fare
    FROM trips
    GROUP BY pickup_date
),
ranked AS (
    SELECT *,
        RANK() OVER (ORDER BY trip_count DESC) AS busiest
    FROM daily_stats
)
SELECT * FROM ranked WHERE busiest <= 10;
```

Star Schema DDL — Dimensional modeling in PostgreSQL

SQL — Star schema DDL

```sql
-- Dimension: date
CREATE TABLE dim_date (
    date_key     INT PRIMARY KEY,
    full_date    DATE NOT NULL,
    year         SMALLINT,
    quarter      SMALLINT,
    month        SMALLINT,
    day_of_week  VARCHAR(10),
    is_weekend   BOOLEAN
);

-- Dimension: location
CREATE TABLE dim_location (
    location_key  SERIAL PRIMARY KEY,
    zone_name     VARCHAR(100),
    borough       VARCHAR(50),
    service_zone  VARCHAR(50)
);

-- Fact: trips (grain = one taxi trip)
CREATE TABLE fact_trips (
    trip_key         BIGSERIAL PRIMARY KEY,
    date_key         INT REFERENCES dim_date,
    pickup_loc_key   INT REFERENCES dim_location,
    dropoff_loc_key  INT REFERENCES dim_location,
    passenger_count  SMALLINT,
    trip_distance    NUMERIC(8,2),
    fare_amount      NUMERIC(8,2),
    tip_amount       NUMERIC(8,2),
    total_amount     NUMERIC(8,2)
);

CREATE INDEX idx_fact_date ON fact_trips(date_key);
CREATE INDEX idx_fact_pickup ON fact_trips(pickup_loc_key);
```

EXPLAIN ANALYZE — Query performance analysis

SQL — EXPLAIN ANALYZE

```sql
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT
    d.full_date,
    l.borough,
    COUNT(*) AS trips,
    AVG(f.fare_amount) AS avg_fare
FROM fact_trips f
JOIN dim_date d ON f.date_key = d.date_key
JOIN dim_location l ON f.pickup_loc_key = l.location_key
WHERE d.year = 2024 AND d.quarter = 1
GROUP BY d.full_date, l.borough
ORDER BY trips DESC;
```

SCD Type 2 — Track dimension history with effective dates

SQL — SCD Type 2

```sql
-- SCD Type 2: keep full history of dimension changes
CREATE TABLE dim_location_scd2 (
    location_sk    SERIAL PRIMARY KEY,   -- surrogate key
    location_id    INT NOT NULL,          -- natural key
    zone_name      VARCHAR(100),
    borough        VARCHAR(50),
    service_zone   VARCHAR(50),
    effective_from DATE NOT NULL,
    effective_to   DATE DEFAULT '9999-12-31',
    is_current     BOOLEAN DEFAULT TRUE
);

-- Close old record and insert new one on change
UPDATE dim_location_scd2
SET effective_to = CURRENT_DATE - 1,
    is_current = FALSE
WHERE location_id = 42 AND is_current = TRUE;

INSERT INTO dim_location_scd2
    (location_id, zone_name, borough, service_zone, effective_from)
VALUES (42, 'New Name', 'Manhattan', 'Yellow Zone', CURRENT_DATE);
```

Materialized Views — Pre-computed query results in PostgreSQL

SQL — Materialized views

```sql
CREATE MATERIALIZED VIEW mv_daily_borough_stats AS
SELECT
    d.full_date,
    l.borough,
    COUNT(*)             AS trip_count,
    ROUND(AVG(f.fare_amount), 2) AS avg_fare,
    ROUND(AVG(f.trip_distance), 2) AS avg_distance
FROM fact_trips f
JOIN dim_date d ON f.date_key = d.date_key
JOIN dim_location l ON f.pickup_loc_key = l.location_key
GROUP BY d.full_date, l.borough;

CREATE UNIQUE INDEX idx_mv_daily_borough
    ON mv_daily_borough_stats (full_date, borough);

-- Refresh without locking reads
REFRESH MATERIALIZED VIEW CONCURRENTLY mv_daily_borough_stats;
```

JSONB Queries — Semi-structured data in PostgreSQL

SQL — JSONB

```sql
-- Store flexible metadata as JSONB
CREATE TABLE events (
    id BIGSERIAL PRIMARY KEY,
    event_type VARCHAR(50),
    payload JSONB NOT NULL,
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Query nested fields
SELECT
    payload->>'user_id' AS user_id,
    payload->'location'->>'city' AS city,
    (payload->>'amount')::numeric AS amount
FROM events
WHERE payload @> '{"status": "completed"}'
  AND created_at > NOW() - INTERVAL '1 day';

-- GIN index for fast JSONB lookups
CREATE INDEX idx_events_payload ON events USING GIN (payload);
```

WEEK 3

Docker + Local Data Stack

Summary

Containerization is non-negotiable in modern data engineering. This week you'll learn Docker from first principles: images, containers, volumes, networks, and multi-stage builds. Then you'll use Docker Compose to orchestrate a multi-container local data stack— the same pattern used in production. This is where most learners fall behind because they skip straight to cloud services. Resist that urge. Understanding containers deeply means you can debug deployment issues, optimize images, and work with any orchestration platform (Kubernetes, ECS, etc.) later.

Key Topics

- Docker fundamentals: images, containers, layers, caching
- Dockerfile best practices: multi-stage builds, minimal base images
- Docker volumes: named volumes, bind mounts, data persistence
- Docker networking: bridge networks, service discovery, port mapping
- Docker Compose: service definitions, dependencies, health checks
- Environment management: .env files, secrets, config injection
- Image optimization: layer caching, .dockerignore, slim images
- Debugging containers: logs, exec, inspect, resource limits

Resources

- **Docker Getting Started** https://docs.docker.com/get-started/
- **Docker Compose Docs** https://docs.docker.com/compose/

GOAL: Build a complete local data infrastructure using containers— understand that every production data platform runs on this foundation.

DELIVERABLES

- ☑ Custom Dockerfiles for Python services (multi-stage, non-root user)
- ☑ docker-compose.yml with PostgreSQL, Airflow, and pgAdmin
- ☑ Health checks and dependency ordering between services
- ☑ Persistent volumes for database data across restarts
- ☑ README documenting how to bring the stack up and down

Week 3 Deep-Dive: Docker Code Examples

Multi-Stage Dockerfile — Slim production images

Dockerfile

```
# Stage 1: build dependencies
FROM python:3.11-slim AS builder
WORKDIR /app
COPY pyproject.toml .
RUN pip install --no-cache-dir --prefix=/install .

# Stage 2: runtime
FROM python:3.11-slim
RUN useradd -m appuser
COPY --from=builder /install /usr/local
COPY src/ /app/src/
WORKDIR /app
USER appuser
EXPOSE 8000
CMD ["uvicorn", "src.app:app", "--host", "0.0.0.0", "--port", "8000"]
```

Docker Compose Stack — Multi-container data platform

docker-compose.yml

```
services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: warehouse
      POSTGRES_USER: de_user
      POSTGRES_PASSWORD: ${PG_PASSWORD}
    volumes:
      - pg_data:/var/lib/postgresql/data
    ports: ["5432:5432"]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U de_user"]
      interval: 5s
      retries: 5

  pgadmin:
    image: dpage/pgadmin4:latest
    environment:
      PGADMIN_DEFAULT_EMAIL: admin@local.dev
      PGADMIN_DEFAULT_PASSWORD: admin
    ports: ["8080:80"]
    depends_on:
      postgres: { condition: service_healthy }

  ingestion:
    build: .
    environment:
      DATABASE_URL: postgresql://de_user:${PG_PASSWORD}@postgres/warehouse
    depends_on:
      postgres: { condition: service_healthy }

volumes:
  pg_data:
```

.dockerignore — Keep images lean

.dockerignore

```
__pycache__/
*.pyc
.venv/
.git/
.env
data/
*.md
tests/
```

Architecture Diagram: Container Stack

```
+-----------------------------------------------+
|              Docker Compose Network            |
|                                                |
|  +-----------+  +-----------+  +--------------+  |
|  | postgres  |  | pgadmin   |  |  ingestion   |  |
|  | :5432     |  | :80       |  |  :8000       |  |
|  |           |  |           |  |              |  |
|  | pg_data   |  | (web UI)  |  | FastAPI app  |  |
|  | volume    |  |           |  | downloads +  |  |
|  |           |  |           |  | loads data   |  |
|  +-----------+  +-----------+  +--------------+  |
|       ^               |               |           |
|       |     connects  |   writes to   |           |
|       +---------------+---------------+           |
+-----------------------------------------------+
       :5432           :8080           :8000
   (host ports exposed to localhost)
```

Health Checks — Ensure services are ready before dependents start

YAML / Dockerfile — health checks

```yaml
# In docker-compose.yml
services:
  postgres:
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U de_user -d warehouse"]
      interval: 5s
      timeout: 3s
      retries: 5
      start_period: 10s

  app:
    depends_on:
      postgres:
        condition: service_healthy

# In Dockerfile (HTTP health check)
HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
    CMD curl -f http://localhost:8000/health || exit 1
```

Docker Networking — How containers find each other

YAML — docker-compose networking

```yaml
# Custom network with explicit subnet
networks:
  data-net:
    driver: bridge
    ipam:
      config:
        - subnet: 172.28.0.0/16

services:
  postgres:
    networks: [data-net]
  app:
    networks: [data-net]
    # connect using service name as hostname:
    # postgres://de_user:pass@postgres:5432/warehouse
    # NOT localhost! Services use container names.
```

Volume Management — Persist data across container restarts

YAML / Bash — volumes

```yaml
# Named volume (Docker manages location)
volumes:
  pg_data:

services:
  postgres:
    volumes:
      - pg_data:/var/lib/postgresql/data    # named volume
      - ./init.sql:/docker-entrypoint-initdb.d/init.sql  # bind mount

# Inspect volume
# docker volume inspect pg_data

# Backup a volume
# docker run --rm -v pg_data:/data -v $(pwd):/backup \
#     alpine tar czf /backup/pg_backup.tar.gz /data
```

WEEK 4

Orchestration + ETL with Apache Airflow

Summary

Orchestration is what turns ad-hoc scripts into reliable data pipelines. Apache Airflow is the industry standard— used at Airbnb, Uber, Spotify, and thousands of companies. This week you'll learn to think in DAGs (Directed Acyclic Graphs): decomposing pipelines into tasks with clear dependencies, retries, and scheduling. You'll understand operators (BashOperator, PythonOperator, PostgresOperator), sensors for event-driven triggers, XCom for inter-task communication, and the critical distinction between orchestrating work vs. doing work inside Airflow. The NYC Taxi dataset is large enough to simulate real-world pipeline challenges.

Key Topics

- Airflow architecture: scheduler, webserver, executor, metadata DB
- DAG design: task dependencies, trigger rules, branching
- Operators: BashOperator, PythonOperator, PostgresOperator, custom operators
- Sensors: file sensors, external task sensors, event-driven triggers
- XCom: inter-task data passing (and when NOT to use it)
- Scheduling: cron expressions, catchup, backfill strategies
- Connections and Hooks: managing external system credentials
- Best practices: idempotency, atomicity, incremental loads

Resources

- **Apache Airflow Docs** https://airflow.apache.org/docs/
- **Astronomer Learn Airflow** https://www.astronomer.io/guides/
- **NYC Taxi Trip Data** https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page

GOAL: Build a production-style ETL pipeline that downloads, transforms, and loads data on a schedule— with retries, logging, and monitoring.

DELIVERABLES

- ☑ Airflow DAG that downloads NYC Taxi data (monthly partitions)
- ☑ Transformation tasks: data cleaning, type casting, deduplication with Pandas
- ☑ Load task: insert into PostgreSQL with upsert logic
- ☑ Retry and alerting configuration for failure scenarios
- ☑ Backfill demonstration: re-run historical months on demand

Week 4 Deep-Dive: Airflow Code Examples

Complete ETL DAG — Download -> Transform -> Load pipeline

Python — dags/taxi_etl.py (part 1)

```python
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.providers.postgres.operators.postgres import PostgresOperator
from datetime import datetime, timedelta
import pandas as pd
import httpx

default_args = {
    "owner": "data-eng",
    "retries": 3,
    "retry_delay": timedelta(minutes=5),
    "retry_exponential_backoff": True,
    "email_on_failure": True,
}

def download_taxi_data(**ctx):
    ds = ctx["ds"]  # e.g. "2024-01-01"
    year, month = ds[:4], ds[5:7]
    url = (f"https://d37ci6vzurychx.cloudfront.net/"
           f"trip-data/yellow_tripdata_{year}-{month}.parquet")
    path = f"/tmp/taxi_{year}_{month}.parquet"
    resp = httpx.get(url, timeout=120)
    resp.raise_for_status()
    with open(path, "wb") as f:
        f.write(resp.content)
    return path

def transform(**ctx):
    path = ctx["ti"].xcom_pull(task_ids="download")
    df = pd.read_parquet(path)
    df = df.dropna(subset=["fare_amount"])
    df = df[df["fare_amount"] > 0]
    out = path.replace(".parquet", "_clean.parquet")
    df.to_parquet(out, index=False)
    return out
```

Python — dags/taxi_etl.py (part 2)

```python
with DAG(
    dag_id="taxi_etl_monthly",
    default_args=default_args,
    schedule="@monthly",
    start_date=datetime(2024, 1, 1),
    catchup=True,
    max_active_runs=2,
    tags=["taxi", "etl"],
) as dag:

    download = PythonOperator(
        task_id="download",
        python_callable=download_taxi_data,
    )

    clean = PythonOperator(
        task_id="transform",
        python_callable=transform,
    )

    create_table = PostgresOperator(
        task_id="create_table",
        postgres_conn_id="warehouse",
        sql="""
            CREATE TABLE IF NOT EXISTS taxi_trips (
                pickup_datetime TIMESTAMP,
                dropoff_datetime TIMESTAMP,
                passenger_count INT,
                trip_distance FLOAT,
                fare_amount FLOAT
            );
        """,
    )

    download >> clean >> create_table
```

Architecture Diagram: Airflow DAG Flow

```
+------------+     +-------------+     +--------------+
|  download  | --> |  transform  | --> | create_table |
|            |     |             |     |              |
| httpx GET  |     | pandas clean |     | PostgresOp   |
| .parquet   |     | drop nulls  |     | CREATE TABLE |
+------------+     +-------------+     +--------------+
      |                  |                   |
      v                  v                   v
XCom: file path     XCom: clean path     SQL executed

Schedule: @monthly    Retries: 3    Catchup: enabled
Max active runs: 2   Backfill: airflow dags backfill
```

Airflow Connections — Managing external credentials

Bash — connection setup

```bash
# Set via CLI (preferred for automation)
airflow connections add warehouse \
    --conn-type postgres \
    --conn-host postgres \
    --conn-port 5432 \
    --conn-login de_user \
    --conn-password "${PG_PASSWORD}" \
    --conn-schema warehouse

# Or via environment variable
export AIRFLOW_CONN_WAREHOUSE=\
    "postgresql://de_user:pass@postgres:5432/warehouse"
```

Backfill — Re-run historical DAG runs

Bash

```bash
# Backfill 6 months of taxi data
airflow dags backfill taxi_etl_monthly \
    --start-date 2024-01-01 \
    --end-date 2024-06-30 \
    --reset-dagruns
```

XCom: Inter-Task Communication — Pass data between tasks

Python — XCom patterns

```python
# Push a value (implicit via return)
def extract(**ctx):
    path = "/tmp/data_2024_01.parquet"
    download(path)
    return path  # auto-pushed to XCom

# Pull a value in another task
def transform(**ctx):
    path = ctx["ti"].xcom_pull(task_ids="extract")
    df = pd.read_parquet(path)
    # ... transform ...

# Push multiple values with explicit keys
def extract_multi(**ctx):
    ctx["ti"].xcom_push(key="row_count", value=10_000)
    ctx["ti"].xcom_push(key="file_path", value="/tmp/data.parquet")

# Pull specific key
def load(**ctx):
    count = ctx["ti"].xcom_pull(
        task_ids="extract_multi", key="row_count")

# WARNING: XCom is stored in metadata DB
# Never pass large data (DataFrames, files)
# Pass file paths or S3 URIs instead
```

Sensor: Wait for File — Event-driven pipeline triggering

Python — sensor DAG

```python
from airflow.sensors.filesystem import FileSensor
from airflow.operators.python import PythonOperator

wait_for_file = FileSensor(
    task_id="wait_for_data",
    filepath="/data/incoming/trades_{{ ds }}.csv",
    poke_interval=60,     # check every 60 seconds
    timeout=3600,         # give up after 1 hour
    mode="reschedule",    # free worker slot between checks
)

process = PythonOperator(
    task_id="process_data",
    python_callable=process_trades,
)

wait_for_file >> process
```

Custom Operator — Encapsulate reusable pipeline logic

Python — custom operator

```python
from airflow.models import BaseOperator
from airflow.utils.context import Context
import httpx
from pathlib import Path

class DownloadOperator(BaseOperator):
    """Download a file from a URL and save locally."""

    template_fields = ("url", "dest_path")

    def __init__(self, url: str, dest_path: str, **kwargs):
        super().__init__(**kwargs)
        self.url = url
        self.dest_path = dest_path

    def execute(self, context: Context):
        self.log.info(f"Downloading {self.url}")
        resp = httpx.get(self.url, timeout=120)
        resp.raise_for_status()
        Path(self.dest_path).write_bytes(resp.content)
        self.log.info(f"Saved to {self.dest_path}")
        return self.dest_path
```

WEEK 5

Modern ELT with dbt (Data Build Tool)

Summary

dbt has revolutionized analytics engineering by bringing software engineering practices to SQL transformations. Instead of writing monolithic stored procedures, you write modular SQL models with version control, testing, and documentation built in. This week you'll learn the ELT paradigm (Extract-Load first, then Transform in the warehouse) and build a full dbt project: staging models that clean raw data, intermediate models for business logic, and mart models optimized for analysts. You'll add schema tests (not_null, unique, relationships), write custom macros with Jinja, and generate a documentation site that makes your data warehouse self-describing.

Key Topics

- ELT vs ETL: why modern stacks transform in the warehouse
- dbt project structure: models, tests, macros, seeds, snapshots
- Model materialization: view, table, incremental, ephemeral
- Staging -> Intermediate -> Mart layering pattern
- Testing: schema tests, data tests, custom test macros
- Jinja templating: macros, control structures, dynamic SQL
- Documentation: model descriptions, column docs, auto-generated site
- Snapshots: tracking slowly changing dimensions with dbt

Resources

- **dbt Documentation** https://docs.getdbt.com/docs/introduction
- **dbt Fundamentals (Free Course)** https://courses.getdbt.com/

GOAL: Build a complete dbt project that transforms raw data into clean, tested, documented analytics models— the core of modern analytics engineering.

DELIVERABLES

- ☑ dbt project: raw → staging → intermediate → marts layer structure
- ☑ 5+ models with proper materializations (view for staging, table for marts)
- ☑ Schema tests: not_null, unique, accepted_values, relationships
- ☑ Custom Jinja macro for a reusable transformation pattern
- ☑ Generated dbt docs site with model lineage graph

Week 5 Deep-Dive: dbt Code Examples

dbt_project.yml — Project-level configuration

YAML — dbt_project.yml

```yaml
name: taxi_analytics
version: "1.0.0"
profile: taxi

model-paths: ["models"]
test-paths: ["tests"]
macro-paths: ["macros"]
seed-paths: ["seeds"]

models:
  taxi_analytics:
    staging:
      +materialized: view
      +schema: staging
    intermediate:
      +materialized: ephemeral
    marts:
      +materialized: table
      +schema: analytics
```

Staging Model — Clean raw data with consistent naming

SQL — stg_taxi_trips.sql

```sql
-- models/staging/stg_taxi_trips.sql
WITH source AS (
    SELECT * FROM {{ source("raw", "taxi_trips") }}
),
renamed AS (
    SELECT
        tpep_pickup_datetime   AS pickup_at,
        tpep_dropoff_datetime  AS dropoff_at,
        passenger_count,
        trip_distance,
        fare_amount,
        tip_amount,
        total_amount,
        payment_type,
        pulocationid           AS pickup_zone_id,
        dolocationid           AS dropoff_zone_id
    FROM source
    WHERE fare_amount > 0
      AND trip_distance > 0
)
SELECT * FROM renamed
```

Mart Model — Business-ready aggregation

SQL — fct_daily_trips.sql

```sql
-- models/marts/fct_daily_trips.sql
WITH trips AS (
    SELECT * FROM {{ ref("stg_taxi_trips") }}
)
SELECT
    DATE_TRUNC('day', pickup_at) AS trip_date,
    pickup_zone_id,
    COUNT(*)               AS total_trips,
    AVG(fare_amount)       AS avg_fare,
    AVG(trip_distance)     AS avg_distance,
    SUM(tip_amount)        AS total_tips,
    AVG(tip_amount / NULLIF(fare_amount, 0))
                           AS avg_tip_pct
FROM trips
GROUP BY 1, 2
```

Schema Tests — Automated data quality in dbt

YAML — schema.yml

```yaml
# models/staging/schema.yml
version: 2

sources:
  - name: raw
    schema: public
    tables:
      - name: taxi_trips
        loaded_at_field: tpep_pickup_datetime
        freshness:
          warn_after: {count: 24, period: hour}
          error_after: {count: 48, period: hour}

models:
  - name: stg_taxi_trips
    columns:
      - name: pickup_at
        tests: [not_null]
      - name: fare_amount
        tests:
          - not_null
          - dbt_utils.accepted_range:
              min_value: 0
              max_value: 1000
```

Jinja Macro — Reusable transformation patterns

SQL + Jinja

```sql
-- macros/cents_to_dollars.sql
{% macro cents_to_dollars(column_name) %}
    ROUND({{ column_name }} / 100.0, 2)
{% endmacro %}

-- Usage in a model:
-- SELECT {{ cents_to_dollars("fare_cents") }} AS fare_dollars
```

Incremental Model — Process only new data (not full table)

SQL — incremental model

```sql
-- models/marts/fct_trips_incremental.sql
{{ config(
    materialized="incremental",
    unique_key="trip_id",
    incremental_strategy="merge"
) }}

WITH new_trips AS (
    SELECT * FROM {{ ref("stg_taxi_trips") }}
    {% if is_incremental() %}
    WHERE pickup_at > (
        SELECT MAX(pickup_at) FROM {{ this }}
    )
    {% endif %}
)
SELECT
    {{ dbt_utils.generate_surrogate_key(
        ["pickup_at", "pickup_zone_id", "fare_amount"]
    ) }} AS trip_id,
    *
FROM new_trips
```

Sources and Freshness — Track raw data arrival

YAML — sources.yml

```yaml
# models/staging/sources.yml
version: 2

sources:
  - name: raw
    schema: public
    tables:
      - name: taxi_trips
        loaded_at_field: tpep_pickup_datetime
        freshness:
          warn_after: {count: 24, period: hour}
          error_after: {count: 48, period: hour}
      - name: taxi_zones
        freshness: null  # static reference data

# Check freshness: dbt source freshness
```

dbt Snapshot — SCD Type 2 built into dbt

SQL — dbt snapshot

```
-- snapshots/taxi_zones_snapshot.sql
{% snapshot taxi_zones_snapshot %}
{{ config(
    target_schema="snapshots",
    unique_key="zone_id",
    strategy="check",
    check_cols=["zone_name", "borough"],
) }}

SELECT * FROM {{ source("raw", "taxi_zones") }}

{% endsnapshot %}
```

Architecture Diagram: dbt Model Layers

```
RAW (PostgreSQL)        STAGING (views)         MARTS (tables)

+---------------+     +------------------+     +------------------+
| raw.taxi_trips| --> | stg_taxi_trips   | --> | fct_daily_trips  |
| (as loaded by |     | (renamed cols,   |     | (aggregated by   |
|  Airflow)     |     |  filtered nulls, |     |  date + zone,    |
+---------------+     |  typed)          |     |  avg fare, etc.) |
                      +------------------+     +------------------+
+---------------+     +------------------+     +------------------+
| raw.taxi_zones| --> | stg_taxi_zones   | --> | dim_zone         |
| (lookup table)|     | (renamed, clean) |     | (enriched)       |
+---------------+     +------------------+     +------------------+

Materialization:       view (cheap, always    table (fast queries,
                        fresh)                 refreshed on dbt run)

Tests at every layer: not_null, unique, accepted_values, freshness
```

WEEK 6

Lakehouse Architecture + Apache Spark

Summary

The lakehouse paradigm merges the flexibility of data lakes (cheap storage, any format) with the reliability of data warehouses (ACID transactions, schema enforcement). This week you'll learn Apache Spark for distributed data processing and Delta Lake for building reliable lakehouse tables. You'll understand partitioning strategies, the difference between Parquet and Delta, time travel for data recovery, and how to run Spark SQL analytics on large datasets. The US Flights dataset (millions of rows) is ideal for experiencing Spark's distributed processing power vs. Pandas.

Key Topics

- Spark architecture: driver, executors, DAG scheduler, lazy evaluation
- DataFrames vs RDDs: when to use each (hint: almost always DataFrames)
- Spark SQL: window functions, aggregations, joins at scale
- File formats: CSV → Parquet → Delta (evolution of storage)
- Delta Lake: ACID transactions, schema enforcement/evolution, time travel
- Partitioning strategies: date-based, hash, bucketing trade-offs
- Performance tuning: broadcast joins, caching, partition pruning
- Lakehouse medallion architecture: bronze → silver → gold

Resources

- **Apache Spark Docs** https://spark.apache.org/docs/latest/
- **Delta Lake Documentation** https://docs.delta.io/
- **US Flights Dataset** https://www.kaggle.com/datasets/usdot/flight-delays

GOAL: Build a lakehouse pipeline with Spark and Delta Lake— understand distributed computing and why it matters when data exceeds single-machine capacity.

DELIVERABLES

- ☑ Spark job: ingest raw CSV → Parquet conversion with proper schema
- ☑ Delta Lake tables with partitioning (e.g., by year/month)
- ☑ Time travel demo: query historical versions of Delta tables
- ☑ Spark SQL analytics: top routes, delay patterns, carrier performance
- ☑ Bronze → Silver → Gold medallion pipeline implementation

Week 6 Deep-Dive: Spark + Delta Lake

PySpark: Read + Transform — DataFrame API basics

Python — spark_etl.py

```python
from pyspark.sql import SparkSession
from pyspark.sql import functions as F

spark = SparkSession.builder \
    .appName("taxi-lakehouse") \
    .config("spark.jars.packages",
            "io.delta:delta-spark_2.12:3.1.0") \
    .config("spark.sql.extensions",
            "io.delta.sql.DeltaSparkSessionExtension") \
    .getOrCreate()

# Read raw CSV
raw = spark.read.csv("data/raw/flights.csv",
                     header=True, inferSchema=True)

# Transform
cleaned = (
    raw
    .filter(F.col("DEP_DELAY").isNotNull())
    .withColumn("flight_date",
        F.to_date(F.col("FL_DATE")))
    .withColumn("is_delayed",
        F.when(F.col("DEP_DELAY") > 15, True)
         .otherwise(False))
    .select("flight_date", "CARRIER", "ORIGIN",
            "DEST", "DEP_DELAY", "ARR_DELAY",
            "is_delayed")
)

# Write as Delta Lake table (partitioned)
cleaned.write \
    .format("delta") \
    .partitionBy("flight_date") \
    .mode("overwrite") \
    .save("data/lakehouse/silver/flights")
```

Delta Lake Time Travel — Query historical versions

Python — time_travel.py

```python
# Read the latest version
current = spark.read.format("delta") \
    .load("data/lakehouse/silver/flights")

# Read version 0 (initial load)
v0 = spark.read.format("delta") \
    .option("versionAsOf", 0) \
    .load("data/lakehouse/silver/flights")

# Read as of a timestamp
yesterday = spark.read.format("delta") \
    .option("timestampAsOf", "2024-06-14") \
    .load("data/lakehouse/silver/flights")

# Show version history
from delta.tables import DeltaTable
dt = DeltaTable.forPath(spark, "data/lakehouse/silver/flights")
dt.history().show(truncate=False)
```

Spark SQL Analytics — SQL interface on Delta tables

Python — spark_analytics.py

```python
spark.sql("""
    SELECT
        CARRIER,
        COUNT(*) AS flights,
        ROUND(AVG(DEP_DELAY), 1) AS avg_delay,
        ROUND(SUM(CASE WHEN is_delayed THEN 1 ELSE 0 END)
              * 100.0 / COUNT(*), 1) AS delay_pct
    FROM delta.`data/lakehouse/silver/flights`
    GROUP BY CARRIER
    ORDER BY delay_pct DESC
    LIMIT 10
""").show()
```

Architecture Diagram: Medallion Lakehouse

```
RAW FILES          BRONZE              SILVER              GOLD
(CSV/JSON)       (raw Delta)        (cleaned Delta)    (aggregated)

+----------+     +-----------+     +-------------+    +----------+
| flights  | --> | raw_      | --> | flights     | -> | daily_   |
| .csv     |     | flights   |     | (deduped,   |    | flight_  |
+----------+     | (as-is,   |     |  typed,     |    | stats    |
                 |  append)  |     |  filtered)  |    +----------+
+----------+     +-----------+     +-------------+    +----------+
| weather  | --> | raw_      | --> | weather     | -> | carrier_ |
| .json    |     | weather   |     | (joined,    |    | perf     |
+----------+     +-----------+     |  enriched)  |    +----------+
                                   +-------------+

Storage: local filesystem or S3
Format:  Delta Lake (Parquet + transaction log)
Engine:  Apache Spark 3.5 / Delta 3.1
```

Performance Tuning — Making Spark jobs faster

Python — Spark tuning

```python
# Tune shuffle partitions (default 200 is too many for small data)
spark.conf.set("spark.sql.shuffle.partitions", "50")

# Broadcast small dimension tables for efficient joins
from pyspark.sql.functions import broadcast
joined = flights.join(
    broadcast(airports),  # small table < 10MB
    flights.ORIGIN == airports.IATA,
    "left"
)

# Cache intermediate results used multiple times
silver = spark.read.format("delta") \
    .load("data/lakehouse/silver/flights")
silver.cache()  # keep in memory
silver.count()  # trigger caching

# Partition pruning: filter on partition column
# Spark reads only matching partitions from disk
recent = silver.filter(
    F.col("flight_date") >= "2024-01-01"
)

# Adaptive Query Execution (Spark 3.x default)
spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled", "true")
```

Delta Lake MERGE — Upsert pattern for lakehouse

Python — Delta MERGE + maintenance

```python
from delta.tables import DeltaTable

# Target: existing silver table
target = DeltaTable.forPath(spark, "data/lakehouse/silver/flights")

# Source: new batch of incoming data
new_data = spark.read.parquet("data/raw/flights_latest.parquet")

# MERGE: update existing rows, insert new ones
target.alias("t").merge(
    new_data.alias("s"),
    "t.flight_id = s.flight_id"
).whenMatchedUpdateAll() \
 .whenNotMatchedInsertAll() \
 .execute()

# Compact small files (maintenance)
target.optimize().executeCompaction()

# Remove old versions (save storage)
target.vacuum(168)  # keep 7 days of history
```

WEEK 7

Streaming + Kafka + Data Quality

Summary

Real-time data is everywhere: user clickstreams, IoT sensors, financial transactions, application logs. Apache Kafka is the backbone of streaming infrastructure at LinkedIn, Netflix, and most modern tech companies. This week you'll learn Kafka's architecture (brokers, topics, partitions, consumer groups) and build a producer/consumer pipeline. Then you'll add Great Expectations for data quality— automated validation that catches schema drift, null spikes, and distribution anomalies before bad data reaches your warehouse. Data quality is what separates hobby projects from production systems.

Key Topics

- Kafka architecture: brokers, topics, partitions, replication
- Producers: serialization, partitioning strategies, acks
- Consumers: consumer groups, offsets, exactly-once semantics
- Schema management: Avro/JSON Schema with Schema Registry
- Great Expectations: expectations, suites, checkpoints, data docs
- Custom expectations: domain-specific validation rules
- Data quality in pipelines: inline validation vs. post-load checks
- Alerting on data quality failures: integration with Slack/email

Resources

- **Apache Kafka Docs** https://kafka.apache.org/documentation/
- **Great Expectations Docs** https://docs.greatexpectations.io/

GOAL: Build a streaming pipeline with data quality gates— understand that reliable data is more valuable than fast data.

DELIVERABLES

- ☑ Kafka producer simulating real-time events (e.g., stock trades, web clicks)
- ☑ Kafka consumer writing events to PostgreSQL with error handling
- ☑ Great Expectations suite: 10+ expectations on your dataset
- ☑ Checkpoint integrated into your Airflow DAG (fail pipeline on bad data)
- ☑ Data Docs site: auto-generated validation reports

Week 7 Deep-Dive: Kafka + Data Quality

Kafka Producer — Publish events to a topic

Python — producer.py

```python
from confluent_kafka import Producer
import json
import time
import random

producer = Producer({
    "bootstrap.servers": "localhost:9092",
    "acks": "all",
    "linger.ms": 10,
    "batch.num.messages": 100,
})

SYMBOLS = ["AAPL", "GOOG", "MSFT", "AMZN", "TSLA"]

def generate_trade():
    return {
        "symbol": random.choice(SYMBOLS),
        "price": round(random.uniform(100, 500), 2),
        "quantity": random.randint(1, 1000),
        "timestamp": int(time.time() * 1000),
    }

for _ in range(10_000):
    trade = generate_trade()
    producer.produce(
        topic="trades",
        key=trade["symbol"],
        value=json.dumps(trade),
    )
producer.flush()
```

Kafka Consumer — Read events and write to database

Python — consumer.py

```python
from confluent_kafka import Consumer
import json

consumer = Consumer({
    "bootstrap.servers": "localhost:9092",
    "group.id": "trade-writer",
    "auto.offset.reset": "earliest",
    "enable.auto.commit": False,
})
consumer.subscribe(["trades"])

batch = []
try:
    while True:
        msg = consumer.poll(timeout=1.0)
        if msg is None:
            continue
        if msg.error():
            print(f"Error: {msg.error()}")
            continue
        trade = json.loads(msg.value())
        batch.append(trade)
        if len(batch) >= 500:
            # insert_batch(batch)  # your DB insert
            consumer.commit()
            batch.clear()
finally:
    consumer.close()
```

Architecture Diagram: Kafka Streaming Pipeline

```
+------------+     +-------------------+     +-------------+
|  Producer  | --> |   Kafka Broker    | --> |  Consumer   |
| (Python)   |     |                   |     |  Group A    |
| generate   |     |  topic: trades    |     |  (write to  |
| trade      |     |  partitions: 6    |     |   Postgres) |
| events     |     |  replication: 3   |     +-------------+
+------------+     |                   |     +-------------+
                   |  topic: orders    | --> |  Consumer   |
                   |  partitions: 3    |     |  Group B    |
                   +-------------------+     |  (analytics)|
                          |                  +-------------+
                   +------v---------+
                   | Schema Registry|
                   | (Avro/JSON)    |
                   +----------------+
```

Kafka with Avro Schema — Type-safe serialization

Python — Avro producer

```python
from confluent_kafka.schema_registry import SchemaRegistryClient
from confluent_kafka.schema_registry.avro import AvroSerializer
from confluent_kafka import SerializingProducer

schema_str = """
{
  "type": "record",
  "name": "Trade",
  "fields": [
    {"name": "symbol",    "type": "string"},
    {"name": "price",     "type": "double"},
    {"name": "quantity",  "type": "int"},
    {"name": "timestamp", "type": "long"}
  ]
}
"""

sr = SchemaRegistryClient({"url": "http://localhost:8081"})
avro_ser = AvroSerializer(sr, schema_str)

producer = SerializingProducer({
    "bootstrap.servers": "localhost:9092",
    "value.serializer": avro_ser,
})

producer.produce("trades",
    value={"symbol": "AAPL", "price": 175.50,
           "quantity": 100, "timestamp": 1718000000})
```

Great Expectations Suite — Automated data validation

Python — validate_trades.py

```python
import great_expectations as gx

context = gx.get_context()

# Add a Pandas datasource
ds = context.sources.add_pandas("trades_ds")
asset = ds.add_dataframe_asset("trades_df")

# Build expectation suite
suite = context.add_expectation_suite("trades_quality")

suite.add_expectation(
    gx.expectations.ExpectColumnValuesToNotBeNull(
        column="symbol"))
suite.add_expectation(
    gx.expectations.ExpectColumnValuesToBeBetween(
        column="price", min_value=0.01, max_value=10000))
suite.add_expectation(
    gx.expectations.ExpectColumnValuesToBeBetween(
        column="quantity", min_value=1, max_value=100000))
suite.add_expectation(
    gx.expectations.ExpectColumnValuesToBeInSet(
        column="symbol",
        value_set=["AAPL","GOOG","MSFT","AMZN","TSLA"]))

# Run checkpoint
checkpoint = context.add_or_update_checkpoint(
    name="trades_checkpoint",
    validations=[{
        "batch_request": asset.build_batch_request(),
        "expectation_suite_name": "trades_quality",
    }],
)
result = checkpoint.run()
assert result.success, "Data quality check failed!"
```

Custom Expectation — Domain-specific validation rules

Python — custom expectation

```python
from great_expectations.expectations.expectation import (
    ColumnMapExpectation,
)

class ExpectColumnValuesToBeValidTicker(ColumnMapExpectation):
    """Expect stock ticker symbols to be 1-5 uppercase letters."""

    map_metric = "column_values.match_regex"
    success_keys = ("regex",)

    default_kwarg_values = {
        "regex": r"^[A-Z]{1,5}$",
        "mostly": 1.0,
    }

# Usage:
# suite.add_expectation(
#     ExpectColumnValuesToBeValidTicker(column="symbol")
# )
```

GX in Airflow — Integrate data quality into your DAG

Python — GX in Airflow DAG

```python
# Inside an Airflow PythonOperator
def validate_loaded_data(**ctx):
    import great_expectations as gx
    import pandas as pd

    df = pd.read_sql("SELECT * FROM trades", engine)
    context = gx.get_context()
    result = context.run_checkpoint(
        checkpoint_name="trades_checkpoint",
        batch_request={
            "runtime_parameters": {"batch_data": df},
            "batch_identifiers": {"run_id": ctx["run_id"]},
        },
    )
    if not result.success:
        raise ValueError("Data quality check failed!")

# In DAG: download >> load >> validate >> notify
validate = PythonOperator(
    task_id="validate",
    python_callable=validate_loaded_data,
)
```

WEEK 8

Cloud + CI/CD + Infrastructure as Code

Summary

Everything you've built locally now goes to the cloud. AWS dominates the job market for data engineering, so you'll learn S3 (object storage for your lakehouse), Redshift (cloud data warehouse), and IAM (security). Terraform lets you define all this infrastructure as code— version-controlled, reviewable, and reproducible. Finally, GitHub Actions automates your pipeline: run dbt tests on every PR, lint SQL, validate schemas, and deploy on merge. This week ties everything together into a production-grade, cloud-native data platform.

Key Topics

- AWS S3: buckets, lifecycle policies, storage classes, IAM policies
- AWS Redshift: clusters, COPY command, distribution/sort keys, Spectrum
- IAM: least-privilege policies, roles, service accounts
- Terraform: providers, resources, variables, state management, modules
- Terraform for data infra: S3 buckets, Redshift clusters, IAM roles
- GitHub Actions: workflows, jobs, steps, secrets, matrix builds
- CI for data: dbt test on PR, SQL linting, schema validation
- CD for data: automated deployment of Airflow DAGs and dbt models

Resources

- **AWS Documentation** https://docs.aws.amazon.com/
- **Terraform Documentation** https://developer.hashicorp.com/terraform/docs
- **GitHub Actions Docs** https://docs.github.com/en/actions

GOAL: Deploy your entire data platform to the cloud with infrastructure as code and automated CI/CD— the final step to production readiness.

DELIVERABLES

- ☑ Terraform config: S3 bucket + Redshift cluster + IAM roles
- ☑ Airflow DAG modified to upload data to S3 and COPY into Redshift
- ☑ GitHub Actions CI: run dbt test and Great Expectations on every PR
- ☑ GitHub Actions CD: deploy DAGs and dbt models on merge to main
- ☑ Full architecture diagram documenting the end-to-end platform

Week 8 Deep-Dive: Cloud + CI/CD

Terraform: S3 + Redshift — Infrastructure as code

HCL — main.tf

```
terraform {
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
  }
  backend "s3" {
    bucket = "my-tf-state"
    key    = "data-platform/terraform.tfstate"
    region = "us-east-1"
  }
}

provider "aws" { region = var.region }

resource "aws_s3_bucket" "lakehouse" {
  bucket = "${var.project}-lakehouse"
  tags   = { Environment = var.env }
}

resource "aws_s3_bucket_versioning" "lakehouse" {
  bucket = aws_s3_bucket.lakehouse.id
  versioning_configuration { status = "Enabled" }
}

resource "aws_redshift_cluster" "warehouse" {
  cluster_identifier  = "${var.project}-warehouse"
  database_name       = "analytics"
  master_username     = "admin"
  master_password     = var.redshift_password
  node_type           = "dc2.large"
  number_of_nodes     = 2
  skip_final_snapshot = true
}
```

GitHub Actions CI — Automated testing on every PR

YAML — .github/workflows/ci.yml

```yaml
name: Data Platform CI
on:
  pull_request:
    branches: [main]

jobs:
  lint-and-test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_DB: test_warehouse
          POSTGRES_PASSWORD: test
        ports: ["5432:5432"]
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-retries 5
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.11" }
      - run: pip install -e ".[dev]"
      - run: ruff check .
      - run: pytest tests/ -v --tb=short

  dbt-test:
    runs-on: ubuntu-latest
    needs: lint-and-test
    steps:
      - uses: actions/checkout@v4
      - run: pip install dbt-postgres
      - run: dbt deps
      - run: dbt build --select state:modified+
```

Architecture Diagram: Cloud Data Platform

```
+--------+     +----------+     +----------+     +-----------+
| GitHub | --> | Actions  | --> | Terraform| --> |   AWS     |
| (code) |     | (CI/CD)  |     | (IaC)    |     |           |
+--------+     +----------+     +----------+     |  +-----+ |
                                                  |  | S3  | |
+---------+     +---------+     +----------+     |  | raw | |
| Kafka   | --> | Airflow | --> | Spark    | --> |  +-----+ |
| (stream)|     | (orch)  |     | (proc)   |     |    |     |
+---------+     +---------+     +----------+     |    v     |
                                                  |  +-----+ |
+---------+     +---------+                      |  |Redsh| |
| GX      | --> | dbt     | -------------------> |  |ift  | |
| (qual)  |     | (ELT)   |                      |  +-----+ |
+---------+     +---------+                      |    |     |
                                                  |    v     |
                                                  | +------+ |
                                                  | |Super-| |
                                                  | |set   | |
                                                  | +------+ |
                                                  +-----------+
```

AWS CLI: S3 Operations — Common data engineering S3 commands

Bash / SQL

```bash
# Sync local data to S3
aws s3 sync data/lakehouse/ s3://my-lakehouse/ \
    --exclude "*.tmp"

# List objects with size
aws s3 ls s3://my-lakehouse/silver/flights/ \
    --recursive --human-readable

# Copy from Redshift via UNLOAD
UNLOAD ('SELECT * FROM analytics.fct_daily_trips')
TO 's3://my-lakehouse/exports/daily_trips_'
IAM_ROLE 'arn:aws:iam::123456:role/RedshiftS3'
FORMAT AS PARQUET;
```

GitHub Actions CD — Deploy on merge to main

YAML — .github/workflows/cd.yml

```yaml
name: Deploy Data Platform
on:
  push:
    branches: [main]

jobs:
  deploy-infra:
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
      - run: terraform init
        working-directory: terraform/
      - run: terraform apply -auto-approve
        working-directory: terraform/
        env:
          TF_VAR_redshift_password: ${{ secrets.REDSHIFT_PW }}

  deploy-dbt:
    needs: deploy-infra
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pip install dbt-redshift
      - run: dbt build --target prod
        env:
          DBT_PROFILES_DIR: ./dbt

  deploy-dags:
    needs: deploy-infra
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: |
          aws s3 sync dags/ s3://my-airflow-bucket/dags/ \
              --delete
```

Terraform Variables + Outputs — Parameterize your infrastructure

HCL — variables.tf + outputs.tf

```
# variables.tf
variable "project" {
  type        = string
  description = "Project name prefix"
  default     = "de-platform"
}

variable "env" {
  type    = string
  default = "dev"
}

variable "region" {
  type    = string
  default = "us-east-1"
}

variable "redshift_password" {
  type      = string
  sensitive = true
}

# outputs.tf
output "s3_bucket_name" {
  value = aws_s3_bucket.lakehouse.bucket
}

output "redshift_endpoint" {
  value = aws_redshift_cluster.warehouse.endpoint
}

output "redshift_iam_role_arn" {
  value = aws_iam_role.redshift_s3.arn
}
```

Redshift COPY + Spectrum — Load and query S3 data

SQL — Redshift COPY + Spectrum

```sql
-- COPY: bulk load from S3 into Redshift
COPY analytics.fact_trips
FROM 's3://my-lakehouse/gold/daily_trips/'
IAM_ROLE 'arn:aws:iam::123456:role/RedshiftS3'
FORMAT AS PARQUET;

-- Redshift Spectrum: query S3 directly (no COPY needed)
CREATE EXTERNAL SCHEMA spectrum_lake
FROM DATA CATALOG
DATABASE 'lakehouse_db'
IAM_ROLE 'arn:aws:iam::123456:role/RedshiftSpectrum';

-- Query S3 data as if it were a local table
SELECT
    flight_date,
    COUNT(*) AS flights,
    AVG(dep_delay) AS avg_delay
FROM spectrum_lake.silver_flights
WHERE flight_date >= '2024-01-01'
GROUP BY 1;
```

IAM Least-Privilege Policy — Secure access for Redshift to S3

HCL — IAM policy

```hcl
resource "aws_iam_role_policy" "redshift_s3_access" {
  name = "${local.prefix}-redshift-s3-policy"
  role = aws_iam_role.redshift_s3.id

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Effect = "Allow"
        Action = [
          "s3:GetObject",
          "s3:ListBucket",
          "s3:GetBucketLocation",
        ]
        Resource = [
          aws_s3_bucket.lakehouse.arn,
          "${aws_s3_bucket.lakehouse.arn}/*",
        ]
      }
    ]
  })
}
```

Public Datasets for Practice

These datasets are freely available and large enough to simulate real-world data engineering challenges. Use them throughout the 8 weeks.

NYC Taxi Trip Data

https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page
Millions of taxi/rideshare records per month. Ideal for ETL, partitioning, and analytics.

Kaggle Datasets

https://www.kaggle.com/datasets
Thousands of curated datasets across domains. Great for exploration and prototyping.

UCI Machine Learning Repository

https://archive.ics.uci.edu/
Classic datasets for statistical analysis and data quality exercises.

Google BigQuery Public Datasets

https://cloud.google.com/bigquery/public-data
Massive-scale datasets (GitHub, Stack Overflow, Wikipedia) queryable for free.

GitHub Archive

https://www.gharchive.org/
Every public GitHub event since 2011. Perfect for streaming and time-series practice.

World Bank Open Data

https://data.worldbank.org/
Global economic indicators. Great for dimensional modeling and reporting.

Recommended Reading

These four books form the intellectual foundation for modern data engineering. Read them in parallel with the hands-on work— theory reinforces practice.

Designing Data-Intensive Applications

by Martin Kleppmann

The "bible" of distributed systems for data engineers. Covers replication, partitioning, consistency, batch/stream processing, and the fundamental trade-offs behind every technology in your stack. Read chapters alongside the corresponding week (e.g., Kafka chapters during Week 7).

Fundamentals of Data Engineering

by Joe Reis & Matt Housley

A modern overview of the entire data engineering landscape: ingestion, storage, transformation, serving, and orchestration. Excellent for building mental models of how all the tools fit together.

The Data Warehouse Toolkit

by Ralph Kimball & Margy Ross

The definitive guide to dimensional modeling. Essential for Week 2's data modeling work and for understanding why star schemas power every BI platform. Focus on the first 8 chapters.

Streaming Systems

by Tyler Akidau, Slava Chernyak & Reuven Lax

Deep dive into stream processing theory: watermarks, windows, triggers, and exactly-once semantics. Pairs perfectly with Week 7's Kafka work. Written by the creators of Google Dataflow.

Weekly Study Plan Template

Use this template to structure your 15 hours each week. Adapt it to your schedule, but stick to the ratio: 30% learning, 50% building, 20% reading.

Monday-Tuesday: Learn (5 hrs)

- Watch tutorials / read documentation for the week's core tool
- Follow along with official quickstart guides
- Take notes on key concepts and architecture decisions

Wednesday-Friday: Build (7 hrs)

- Set up the tool locally (Docker, pip install, etc.)
- Implement the week's deliverables step by step
- Write tests as you build (not after)
- Commit working code to Git daily

Weekend: Read + Reflect (3 hrs)

- Read the corresponding book chapters (see Recommended Reading)
- Review and refactor your code from the week
- Write a brief README for what you built
- Plan next week's focus areas

Progress Tracking

```
Week  Mon   Tue   Wed   Thu   Fri   Sat   Sun   Done?
----  ----  ----  ----  ----  ----  ----  ----  -----
  1   [ ]   [ ]   [ ]   [ ]   [ ]   [ ]   [ ]   [ ]
  2   [ ]   [ ]   [ ]   [ ]   [ ]   [ ]   [ ]   [ ]
  3   [ ]   [ ]   [ ]   [ ]   [ ]   [ ]   [ ]   [ ]
  4   [ ]   [ ]   [ ]   [ ]   [ ]   [ ]   [ ]   [ ]
  5   [ ]   [ ]   [ ]   [ ]   [ ]   [ ]   [ ]   [ ]
  6   [ ]   [ ]   [ ]   [ ]   [ ]   [ ]   [ ]   [ ]
  7   [ ]   [ ]   [ ]   [ ]   [ ]   [ ]   [ ]   [ ]
  8   [ ]   [ ]   [ ]   [ ]   [ ]   [ ]   [ ]   [ ]
```

Tips for Success

- Don't skip Docker week. Every production issue traces back to containers.
- Write tests from day one. Testing isn't optional in data engineering.
- Use Git branches for each week's project. Practice the workflow.
- If stuck for > 30 minutes, check Stack Overflow or official Discord.
- Focus on understanding WHY, not just HOW. Interview questions test depth.

Final Portfolio Project

After 8 weeks, you'll combine everything into a single end-to-end project that demonstrates your capabilities to hiring managers. This is your portfolio centerpiece.

Architecture

Kafka (streaming ingestion) → Airflow (orchestration) → S3 (raw storage) → Spark/Delta Lake (lakehouse processing) → dbt (transformation) → Redshift (serving layer) → Tableau/Superset (visualization)

Cross-cutting concerns: Great Expectations (data quality at every stage), Terraform (all infrastructure as code), GitHub Actions (CI/CD automation).

Capstone Architecture Diagram

```
                        +-------------------+
                        |   Data Sources    |
                        | (APIs, files, DB) |
                        +--------+----------+
                                 |
                    +------------v-----------+
                    |     Apache Kafka       |
                    |  (streaming ingestion) |
                    +------------+-----------+
                                 |
          +----------+-----------+-----------+----------+
          |          |                       |          |
   +-------v---+  +--v--------+      +-------v---+      |
   |  Airflow  |  |    GX     |      |   S3      |      |
   |  (orch)   |  | (quality) |      | (storage) |      |
   +-------+---+  +----------+       +-----+-----+      |
          |                              |             |
          +-------->  Spark / Delta  <----+             |
                    (bronze>silver>gold)                |
                           |                            |
                    +------v------+             +------v------+
                    |     dbt     |             |  Terraform  |
                    | (transform) |             |   (IaC)     |
                    +------+------+             +-------------+
                           |
                    +------v------+
                    |  Redshift   |
                    |  (serving)  |
                    +------+------+
                           |
                    +------v------+
                    |  Superset   |
                    | (dashboards)|
                    +-------------+
```

Example: End-to-End Pipeline Commands

Bash — end-to-end pipeline

```bash
# 1. Start infrastructure
docker compose up -d
terraform init && terraform apply

# 2. Start Kafka producer
python producer.py --topic trades --rate 1000/sec

# 3. Start Kafka consumer (writes to S3)
python consumer.py --topic trades --sink s3://lakehouse/bronze/

# 4. Run Spark ETL (bronze -> silver -> gold)
spark-submit --master local[4] etl/medallion.py

# 5. Run dbt transforms
cd dbt/ && dbt build --target prod

# 6. Run data quality checks
great_expectations checkpoint run trades_checkpoint

# 7. Load into Redshift
psql $REDSHIFT_URL -f sql/copy_gold_to_redshift.sql

# 8. All of this is scheduled via Airflow DAG
# and tested via GitHub Actions CI/CD
```

Portfolio README Template

Markdown — README template

```markdown
# Project Name: Real-Time Trading Analytics Platform

## Architecture
[Include the ASCII diagram from this guide]

## Tech Stack
Python | Kafka | Spark | Delta Lake | dbt | Airflow
PostgreSQL | Redshift | Terraform | GitHub Actions | GX

## Quick Start
```bash
git clone https://github.com/you/project.git
cp .env.example .env
docker compose up -d
make setup  # install deps, init DBs, seed data
make run    # start all pipeline components
```

Data Flow
1. Kafka producer generates simulated trade events
2. Consumer writes raw events to S3 (bronze)
3. Spark cleans, deduplicates, enriches (silver)
4. dbt aggregates into analytics tables (gold)
5. GX validates data quality at each stage
6. Airflow orchestrates the entire pipeline
7. Superset dashboards visualize key metrics
```

What to Include in Your GitHub Repo

- Professional README with architecture diagram
- Docker Compose for local development
- Terraform configs for cloud deployment
- Airflow DAGs with clear documentation
- dbt project with tests and generated docs
- Great Expectations suites and data docs
- GitHub Actions workflows (CI + CD)
- Screenshots of dashboards and monitoring
- Setup instructions that actually work (test them!)

Realistic Outcomes After 8 Weeks

You won't be a "master"— that takes years of production experience. But you will:

- Understand and work with the modern data stack end-to-end
- Deploy reliable batch and streaming data pipelines
- Use infrastructure as code for reproducible environments
- Apply data quality practices that prevent production incidents
- Be interview-ready for mid-level data engineering roles
- Have a portfolio project that demonstrates real engineering skills

Interview Prep: Common Questions

These questions come up repeatedly in data engineering interviews. Use your project experience to answer with concrete examples.

System Design

- Design a real-time analytics pipeline for e-commerce clickstream data.
- How would you build a data lake that supports both batch and streaming?
- Design a pipeline that processes 1TB of data daily with SLA guarantees.
- How do you handle schema evolution in a production data pipeline?

SQL & Modeling

- Explain the difference between star schema and snowflake schema.
- Write a query to find the top 3 products per category by revenue.
- What are SCD Type 1 vs Type 2? When would you use each?
- How do you optimize a slow query? Walk through your approach.

Data Quality & Operations

- How do you ensure data quality in a production pipeline?
- A pipeline failed at 3 AM. Walk through your incident response.
- How do you handle late-arriving data in a streaming pipeline?
- Explain exactly-once semantics. Is it really possible?

Architecture & Trade-offs

- ETL vs ELT: when would you choose each approach?
- Batch vs streaming: what factors drive the decision?
- When would you use Spark vs a simpler tool like Pandas?
- How do you decide between managed cloud services and self-hosted?

Sample Answer Framework (STAR)

Interview answer framework

```
Situation: "In my portfolio project, I built a real-time
           stock trading pipeline using Kafka and Spark."

Task:      "I needed to handle 10K events/sec with < 5 min
           end-to-end latency and zero data loss."

Action:    "I configured Kafka with acks=all and
           enable.idempotence=true for exactly-once
           delivery. I used Delta Lake MERGE for upserts
           and Great Expectations checkpoints to catch
           schema drift before data reached the warehouse."

Result:    "The pipeline processed 500K events/day with
           99.9% uptime. Data quality checks caught 3
           schema changes during development that would
           have caused silent data corruption."
```

Quick Reference: When to Use What

A decision guide for choosing the right tool for common data engineering tasks.

Data Storage

`PostgreSQL`	Structured data < 1TB, strong consistency, ACID, complex queries
`S3 / Object Storage`	Any size, any format, cheap, immutable files (Parquet, JSON, CSV)
`Delta Lake / Iceberg`	ACID on object storage, time travel, schema evolution, lakehouse
`Redshift / BigQuery`	Analytical queries on TB-PB scale, columnar, managed

Data Processing

`Pandas`	Single machine, < 10GB, prototyping, quick analysis
`Apache Spark`	Distributed, > 10GB, production ETL, cluster processing
`dbt`	SQL transforms in warehouse, testing, documentation, lineage
`Apache Flink`	True streaming (event-at-a-time), low latency, stateful processing

Orchestration

`Apache Airflow`	Batch scheduling, DAG-based, mature ecosystem, Python-native
`Dagster`	Asset-oriented, type-safe, great testing, modern alternative
`GitHub Actions`	CI/CD, lightweight scheduling, code-triggered workflows

Streaming

`Apache Kafka`	High-throughput message bus, event sourcing, pub/sub at scale
`AWS SQS / SNS`	Simple queuing, managed, no cluster ops, lower throughput

Data Quality

`Great Expectations`	Batch validation, expectation suites, data docs, rich ecosystem
`dbt tests`	SQL-based tests integrated with transform layer

Infrastructure

`Docker`	Local dev, packaging, reproducible environments
`Docker Compose`	Multi-container local stacks, dev/test environments
`Terraform`	Cloud infrastructure as code, reproducible, version-controlled
`Kubernetes`	Production container orchestration (after mastering Docker)

COOKBOOK

Quick-Reference Cheat Sheets

- Python
- SQL
- Docker
- Airflow
- dbt
- Spark
- Kafka
- Terraform
- Git + GitHub Actions
- Great Expectations

Cookbook: Python

`python -m venv .venv`	Create a virtual environment
`source .venv/bin/activate`	Activate venv (macOS/Linux)
`pip install -e '.[dev]'`	Install package in editable mode with dev extras
`pip freeze > requirements.txt`	Pin all installed package versions
`pip install pip-tools` `pip-compile pyproject.toml`	Generate locked requirements from pyproject.toml
`python -m pytest tests/ -v`	Run tests with verbose output
`python -m pytest --cov=src`	Run tests with coverage report
`ruff check . --fix`	Lint and auto-fix Python files
`ruff format .`	Format all Python files
`python -c 'import sys; print(sys.path)'`	Debug module resolution issues

Common Patterns

Python patterns

```python
# Context manager for file handling
from pathlib import Path
data = Path("data.json").read_text()

# Dataclass for typed records
from dataclasses import dataclass
@dataclass
class Trade:
    symbol: str
    price: float
    quantity: int

# Generator for memory-efficient iteration
def read_chunks(path, size=10_000):
    import pandas as pd
    for chunk in pd.read_csv(path, chunksize=size):
        yield chunk

# Retry decorator
from tenacity import retry, stop_after_attempt
@retry(stop=stop_after_attempt(3))
def fetch_data(url):
    return httpx.get(url).raise_for_status()
```

Data Engineering Patterns

Python — DE patterns

```python
# Atomic file writes (no partial files on crash)
import tempfile, shutil
with tempfile.NamedTemporaryFile(
    dir="data/", delete=False, suffix=".parquet"
) as tmp:
    df.to_parquet(tmp.name)
    shutil.move(tmp.name, "data/final.parquet")

# Connection pooling with context manager
from contextlib import contextmanager
from sqlalchemy import create_engine

@contextmanager
def get_engine():
    engine = create_engine(DATABASE_URL,
        pool_size=5, pool_recycle=3600)
    try:
        yield engine
    finally:
        engine.dispose()

# Parallel downloads with asyncio
import asyncio
async def download_all(urls):
    async with httpx.AsyncClient() as c:
        tasks = [c.get(url) for url in urls]
        return await asyncio.gather(*tasks)
```

Pandas for Data Engineering

Python — pandas for DE

```python
# Read large files efficiently
df = pd.read_parquet("data.parquet",
    columns=["id", "amount", "ts"])  # read only needed cols

# Chunked CSV reading
for chunk in pd.read_csv("big.csv", chunksize=100_000):
    process(chunk)

# Deduplication
df = df.drop_duplicates(subset=["id"], keep="last")

# Type-safe loading
dtypes = {"id": "int64", "amount": "float64",
          "status": "category"}
df = pd.read_csv("data.csv", dtype=dtypes,
    parse_dates=["created_at"])

# Efficient groupby + agg
result = df.groupby("category").agg(
    total=("amount", "sum"),
    count=("id", "count"),
    avg=("amount", "mean"),
).reset_index()

# Write to Parquet (always prefer over CSV)
df.to_parquet("output.parquet", index=False,
    engine="pyarrow", compression="snappy")
```

Cookbook: SQL

`SELECT *, ROW_NUMBER()`
`  OVER (PARTITION BY col`
`  ORDER BY ts DESC) rn`
`FROM t`
	Row number per group (dedup with WHERE rn = 1)

`SELECT *, LAG(val) OVER`
`  (ORDER BY ts) AS prev_val`
`FROM t`
	Previous row value (for change detection)

`SELECT *, SUM(amount) OVER`
`  (ORDER BY ts ROWS BETWEEN`
`  UNBOUNDED PRECEDING`
`  AND CURRENT ROW) AS running`
`FROM t`
	Running total / cumulative sum

`WITH cte AS (`
`  SELECT ... FROM t`
`)`
`SELECT * FROM cte`
	Common Table Expression (named subquery)

`CREATE INDEX CONCURRENTLY` `  idx_name ON t(col);`	Non-blocking index creation (Postgres)
`EXPLAIN (ANALYZE, BUFFERS)` `  SELECT ... ;`	Query execution plan with I/O stats
`COPY table FROM '/path/data.csv'` `  WITH (FORMAT csv, HEADER true);`	Bulk load CSV into Postgres

`\dt  -- list tables`
`\d+ tablename -- describe table`
`\timing  -- toggle timing`
	psql meta-commands

`COALESCE(col, 0)`	Replace NULL with a default value
`PERCENTILE_CONT(0.5) WITHIN` `  GROUP (ORDER BY val) AS median`	Median (50th percentile)

Advanced SQL Patterns

SQL — advanced patterns

```sql
-- Deduplicate rows (keep latest per key)
DELETE FROM events a USING events b
WHERE a.id < b.id
  AND a.event_key = b.event_key;

-- Or with ROW_NUMBER:
WITH ranked AS (
    SELECT *, ROW_NUMBER() OVER (
        PARTITION BY event_key
        ORDER BY created_at DESC
    ) AS rn
    FROM events
)
SELECT * FROM ranked WHERE rn = 1;

-- Gap detection (find missing dates)
WITH date_series AS (
    SELECT generate_series(
        '2024-01-01'::date,
        '2024-12-31'::date,
        '1 day'::interval
    )::date AS dt
)
SELECT dt AS missing_date
FROM date_series
LEFT JOIN daily_data d ON d.report_date = dt
WHERE d.report_date IS NULL;

-- Pivot with FILTER (PostgreSQL)
SELECT
    pickup_date,
    COUNT(*) FILTER (WHERE borough = 'Manhattan') AS manhattan,
    COUNT(*) FILTER (WHERE borough = 'Brooklyn') AS brooklyn,
    COUNT(*) FILTER (WHERE borough = 'Queens') AS queens
FROM trips JOIN zones USING (zone_id)
GROUP BY pickup_date;
```

Cookbook: Docker

`docker build -t myapp:v1 .`	Build image from Dockerfile in current dir
`docker build --target builder .`	Build only up to a specific stage
`docker run -d -p 8000:8000 --name api myapp:v1`	Run container in background with port mapping
`docker run --rm -it myapp sh`	Run interactive shell in a disposable container
`docker compose up -d`	Start all services in detached mode
`docker compose down -v`	Stop services and remove volumes
`docker compose logs -f service`	Follow logs for a specific service
`docker exec -it container bash`	Open a shell in a running container
`docker system prune -af`	Remove all unused images, containers, volumes
`docker volume ls` `docker volume inspect vol`	List and inspect volumes
`docker stats`	Live CPU/memory usage for all containers
`docker inspect container	jq '.[0].NetworkSettings'`	Inspect container network config

Dockerfile Best Practices

Dockerfile patterns

```
# 1. Use specific base tags (not :latest)
FROM python:3.11-slim

# 2. Copy dependency files first (layer caching)
COPY pyproject.toml .
RUN pip install --no-cache-dir .

# 3. Copy source code last (changes most often)
COPY src/ src/

# 4. Run as non-root user
RUN useradd -m app
USER app
```

Docker Troubleshooting

Docker troubleshooting

```
# Debug a failing container
docker logs container_name --tail 50
docker inspect container_name | jq '.[0].State'
docker exec -it container_name /bin/sh

# Check resource usage
docker stats --no-stream

# View layer sizes in an image
docker history myimage:v1 --human

# Network debugging
docker network inspect bridge
docker exec container_name ping other_container

# Clean everything (nuclear option)
docker system prune -af --volumes

# Check what is using disk space
docker system df -v
```

Cookbook: Airflow

`airflow db init`	Initialize Airflow metadata database
`airflow db migrate`	Apply pending database migrations
`airflow dags list`	List all discovered DAGs
`airflow dags test dag_id 2024-01-01`	Test a DAG run for a specific date (no DB writes)
`airflow tasks test dag_id task_id 2024-01-01`	Test a single task (no DB writes)
`airflow dags trigger dag_id`	Manually trigger a DAG run
`airflow dags backfill dag_id -s 2024-01-01 -e 2024-06-30`	Backfill DAG runs for a date range
`airflow connections add conn_id --conn-type postgres --conn-host localhost ...`	Create a connection via CLI
`airflow variables set KEY VALUE`	Set an Airflow variable
`airflow users create --role Admin --username admin --email a@b.com --firstname A --lastname B --password pass`	Create an admin user

DAG Patterns

Airflow patterns

```python
# Idempotent task: use UPSERT / MERGE
# instead of INSERT to handle re-runs

# Task branching
from airflow.operators.python import BranchPythonOperator

def choose_branch(**ctx):
    if ctx["ds"] > "2024-06-01":
        return "new_pipeline"
    return "legacy_pipeline"

# Task groups (sub-DAGs replacement)
from airflow.utils.task_group import TaskGroup

with TaskGroup("extract") as extract:
    task_a = PythonOperator(...)
    task_b = PythonOperator(...)

extract >> transform >> load
```

Common Pitfalls

Airflow anti-patterns

```python
# BAD: doing heavy work inside the DAG file
import pandas as pd
df = pd.read_csv("huge.csv")  # runs at DAG parse time!

# GOOD: defer work to task execution
def process(**ctx):
    import pandas as pd  # import inside function
    df = pd.read_csv("huge.csv")

# BAD: passing large data via XCom
# XCom stores in metadata DB (Postgres)
# return big_dataframe  # will serialize entire DF!

# GOOD: pass file paths, not data
def extract(**ctx):
    path = "/tmp/data.parquet"
    download_to(path)
    return path  # small string, not data
```

Airflow Docker Compose

YAML — Airflow docker-compose

```yaml
# Minimal Airflow for local development
services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: airflow
      POSTGRES_USER: airflow
      POSTGRES_PASSWORD: airflow
    volumes: [pg_data:/var/lib/postgresql/data]

  airflow-init:
    image: apache/airflow:2.9.0
    entrypoint: /bin/bash -c
    command: airflow db migrate && airflow users create
      --role Admin --username admin --password admin
      --email admin@local.dev --firstname Admin
      --lastname User
    depends_on: [postgres]

  airflow-webserver:
    image: apache/airflow:2.9.0
    command: airflow webserver
    ports: ["8080:8080"]
    volumes: [./dags:/opt/airflow/dags]
    depends_on: [airflow-init]

  airflow-scheduler:
    image: apache/airflow:2.9.0
    command: airflow scheduler
    volumes: [./dags:/opt/airflow/dags]
    depends_on: [airflow-init]
```

Cookbook: dbt

`dbt init my_project`	Scaffold a new dbt project
`dbt deps`	Install dbt packages from packages.yml
`dbt run`	Run all models
`dbt run --select model_name`	Run a single model
`dbt run --select model_name+`	Run a model and all downstream
`dbt run --select +model_name`	Run a model and all upstream
`dbt test`	Run all schema and data tests
`dbt build`	Run + test in dependency order
`dbt build --select state:modified+`	Build only changed models and downstream
`dbt docs generate` `dbt docs serve`	Generate and serve documentation site
`dbt snapshot`	Capture SCD Type-2 changes
`dbt seed`	Load CSV seed files into warehouse

Jinja Snippets

Jinja patterns

```
-- Conditional logic
{% if target.name == 'prod' %}
    {{ config(materialized="table") }}
{% else %}
    {{ config(materialized="view") }}
{% endif %}

-- Loop over columns
{% set metrics = ["revenue", "cost", "profit"] %}
{% for m in metrics %}
    SUM({{ m }}) AS total_{{ m }}{% if not loop.last %},{% endif %}
{% endfor %}

-- Source reference
SELECT * FROM {{ source("raw", "orders") }}

-- Ref another model
SELECT * FROM {{ ref("stg_orders") }}
```

dbt Test Patterns

SQL — custom dbt tests

```
# Custom data test (tests/assert_positive_revenue.sql)
-- This query should return 0 rows to pass
SELECT order_id, revenue
FROM {{ ref("fct_orders") }}
WHERE revenue < 0

# Custom generic test (tests/generic/test_is_even.sql)
{% test is_even(model, column_name) %}
SELECT {{ column_name }}
FROM {{ model }}
WHERE {{ column_name }} % 2 != 0
{% endtest %}

# Usage in schema.yml:
# columns:
#   - name: quantity
#     tests:
#       - is_even
```

dbt packages.yml

YAML — packages.yml

```yaml
# packages.yml — install community packages
packages:
  - package: dbt-labs/dbt_utils
    version: ">=1.0.0"
  - package: calogica/dbt_expectations
    version: ">=0.10.0"
  - package: dbt-labs/codegen
    version: ">=0.12.0"

# Install with: dbt deps
# Use dbt_utils macros:
# {{ dbt_utils.generate_surrogate_key(["col1", "col2"]) }}
# {{ dbt_utils.star(from=ref("stg_orders")) }}
```

Cookbook: Spark

`spark-submit --master local[*] --packages io.delta:delta-spark etl.py`	Submit a Spark job locally
`spark-submit --master yarn --deploy-mode cluster --num-executors 10 etl.py`	Submit to YARN cluster
`df = spark.read.parquet("s3://..")`	Read Parquet from S3
`df.write.format("delta") .partitionBy("date") .save("path")`	Write partitioned Delta table
`df.repartition(10)`	Control output file count
`df.cache()`	Cache DataFrame in memory
`spark.conf.set( "spark.sql.shuffle.partitions", "200")`	Tune shuffle partitions
`F.broadcast(small_df)`	Broadcast join hint for small tables
`df.explain("formatted")`	Show physical execution plan

Common Transformations

PySpark patterns

```
from pyspark.sql import functions as F

# Filter + add columns
df = df.filter(F.col("amount") > 0) \
       .withColumn("year", F.year("ts")) \
       .withColumn("amount_usd",
           F.col("amount") * F.col("rate"))

# Window function
from pyspark.sql.window import Window
w = Window.partitionBy("user_id").orderBy("ts")
df = df.withColumn("prev_amount", F.lag("amount").over(w))

# Dedup by key (keep latest)
df = df.dropDuplicates(["id"])

# Pivot
pivoted = df.groupBy("date").pivot("category") \
            .agg(F.sum("amount"))
```

Spark SQL Functions Reference

PySpark — function reference

```python
# Date/time functions
F.year("ts"), F.month("ts"), F.dayofweek("ts")
F.date_trunc("month", "ts")   # truncate to month
F.datediff("end_ts", "start_ts")  # days between

# String functions
F.lower("col"), F.upper("col"), F.trim("col")
F.regexp_extract("col", r"(\d+)", 1)
F.split("col", ",")  # returns array

# Null handling
F.coalesce("col1", "col2", F.lit(0))
F.when(F.col("x").isNull(), "missing")
 .otherwise(F.col("x"))

# Aggregation
F.count_distinct("user_id")
F.approx_count_distinct("user_id")  # faster
F.collect_list("tag")   # array of values
F.collect_set("tag")    # unique values
```

Cookbook: Kafka

`kafka-topics.sh --create --topic trades --partitions 6 --replication-factor 3 --bootstrap-server :9092`	Create a topic
`kafka-topics.sh --list --bootstrap-server :9092`	List all topics
`kafka-topics.sh --describe --topic trades --bootstrap-server :9092`	Show topic details (partitions, ISR)
`kafka-console-producer.sh --topic trades --bootstrap-server :9092`	Produce messages from stdin
`kafka-console-consumer.sh --topic trades --from-beginning --bootstrap-server :9092`	Consume all messages from beginning
`kafka-consumer-groups.sh --describe --group mygroup --bootstrap-server :9092`	Show consumer group lag
`kafka-consumer-groups.sh --reset-offsets --to-earliest --group mygroup --topic trades --execute --bootstrap-server :9092`	Reset consumer group offsets

Python Client Patterns

Python — Kafka patterns

```python
# confluent-kafka producer with delivery report
from confluent_kafka import Producer

def delivery_report(err, msg):
    if err:
        print(f"FAILED: {err}")
    else:
        print(f"OK: {msg.topic()}[{msg.partition()}]")

p = Producer({"bootstrap.servers": "localhost:9092"})
p.produce("topic", value=b"hello",
          callback=delivery_report)
p.flush()

# JSON serialization pattern
import json
p.produce("topic",
          key=record["id"].encode(),
          value=json.dumps(record).encode())
```

Consumer Patterns

Python — batch consumer

```python
# Batch consumer with manual commit
from confluent_kafka import Consumer
import json

c = Consumer({
    "bootstrap.servers": "localhost:9092",
    "group.id": "etl-pipeline",
    "auto.offset.reset": "earliest",
    "enable.auto.commit": False,
    "max.poll.interval.ms": 300000,
})
c.subscribe(["trades"])

BATCH_SIZE = 500
batch = []

while True:
    msg = c.poll(1.0)
    if msg is None:
        if batch:  # flush remaining
            write_to_db(batch)
            c.commit()
            batch.clear()
        continue
    if msg.error():
        handle_error(msg.error())
        continue

    batch.append(json.loads(msg.value()))
    if len(batch) >= BATCH_SIZE:
        write_to_db(batch)
        c.commit()  # commit AFTER successful write
        batch.clear()
```

Kafka Configuration Reference

`acks=all`	Wait for all replicas to acknowledge (strongest durability)
`linger.ms=10`	Wait 10ms to batch messages (throughput vs latency)
`compression.type=snappy`	Compress messages (30-50% size reduction)
`enable.idempotence=true`	Exactly-once producer semantics
`max.poll.records=500`	Max messages per consumer poll()
`session.timeout.ms=45000`	Consumer heartbeat timeout

Kafka Docker Compose

YAML — Kafka docker-compose

```yaml
# Minimal Kafka + Schema Registry for local dev
services:
  kafka:
    image: confluentinc/cp-kafka:7.6.0
    environment:
      KAFKA_NODE_ID: 1
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
      KAFKA_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
      KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
      CLUSTER_ID: MkU3OEVBNTcwNTJENDM2Qk
    ports: ["9092:9092"]

  schema-registry:
    image: confluentinc/cp-schema-registry:7.6.0
    depends_on: [kafka]
    environment:
      SCHEMA_REGISTRY_KAFKASTORE_BOOTSTRAP_SERVERS: kafka:9092
      SCHEMA_REGISTRY_HOST_NAME: schema-registry
    ports: ["8081:8081"]
```

Cookbook: Terraform

`terraform init`	Initialize providers and backend
`terraform plan -out=tfplan`	Preview changes (save plan file)
`terraform apply tfplan`	Apply saved plan
`terraform apply -auto-approve`	Apply without interactive prompt
`terraform destroy`	Destroy all managed resources
`terraform state list`	List all resources in state
`terraform state show aws_s3_bucket.lakehouse`	Show details of a specific resource
`terraform import aws_s3_bucket.lakehouse my-bucket-name`	Import existing resource into state
`terraform fmt -recursive`	Format all .tf files
`terraform validate`	Check config syntax and consistency

Common Resource Patterns

HCL patterns

```
# Variables with defaults
variable "env" {
  type    = string
  default = "dev"
}

# Locals for computed values
locals {
  prefix = "${var.project}-${var.env}"
}

# IAM role for Redshift to access S3
resource "aws_iam_role" "redshift_s3" {
  name = "${local.prefix}-redshift-s3"
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Action = "sts:AssumeRole"
      Effect = "Allow"
      Principal = {
        Service = "redshift.amazonaws.com"
      }
    }]
  })
}

# Output values
output "bucket_arn" {
  value = aws_s3_bucket.lakehouse.arn
}
```

Terraform for Data Pipelines

HCL — S3 lifecycle + Redshift

```
# S3 bucket with lifecycle (move old data to Glacier)
resource "aws_s3_bucket_lifecycle_configuration" "lake" {
  bucket = aws_s3_bucket.lakehouse.id

  rule {
    id     = "archive-old-bronze"
    status = "Enabled"
    filter { prefix = "bronze/" }
    transition {
      days          = 90
      storage_class = "GLACIER"
    }
    expiration { days = 365 }
  }
}

# Redshift with IAM role attachment
resource "aws_redshift_cluster" "wh" {
  cluster_identifier = "${local.prefix}-wh"
  database_name      = "analytics"
  node_type          = "dc2.large"
  number_of_nodes    = 2
  master_username    = "admin"
  master_password    = var.redshift_password
  iam_roles = [aws_iam_role.redshift_s3.arn]
}
```

Cookbook: Git + GitHub Actions

`git log --oneline -20`	View last 20 commits (short format)
`git log --graph --all --oneline`	Visual branch history
`git diff --staged`	Show staged changes (before commit)
`git stash push -m 'wip'` `git stash pop`	Stash and restore work-in-progress
`git rebase -i HEAD~3`	Interactively squash/edit last 3 commits
`git cherry-pick <sha>`	Apply a specific commit to current branch

`git bisect start`
`git bisect bad`
`git bisect good <sha>`
	Binary search for a bug-introducing commit

`git reflog`	View all HEAD movements (recover lost commits)
`git worktree add ../fix branch`	Work on multiple branches simultaneously

GitHub Actions Workflow Patterns

YAML — GitHub Actions patterns

```yaml
# Reusable workflow (called by other workflows)
on:
  workflow_call:
    inputs:
      environment:
        type: string
        required: true

# Matrix builds
strategy:
  matrix:
    python: ["3.11", "3.12"]
    os: [ubuntu-latest, macos-latest]

# Cache dependencies
- uses: actions/cache@v4
  with:
    path: ~/.cache/pip
    key: ${{ runner.os }}-pip-${{ hashFiles("**/requirements.txt") }}

# Deploy on push to main
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
```

Git Branching Strategy

Git — branching patterns

```
# Feature branch workflow
git checkout -b feature/add-kafka-consumer main
# ... make changes, commit ...
git push -u origin feature/add-kafka-consumer
# Create PR, get review, merge

# Release branch workflow
git checkout -b release/v1.2.0 main
# Cherry-pick hotfixes:
git cherry-pick <hotfix-sha>
git tag v1.2.0
git push origin v1.2.0

# Useful aliases for .gitconfig
# [alias]
#   lg = log --oneline --graph --all
#   st = status -sb
#   co = checkout
#   unstage = reset HEAD --
```

Cookbook: Great Expectations

`great_expectations init`	Initialize a GX project in current directory
`great_expectations suite new`	Create a new expectation suite interactively
`great_expectations suite edit suite_name`	Edit an existing suite in a Jupyter notebook
`great_expectations checkpoint run my_checkpoint`	Run a checkpoint (validate data)
`great_expectations docs build`	Build and open Data Docs site

Common Expectations

Python — GX expectation patterns

```python
# Column-level expectations
expect_column_values_to_not_be_null("id")
expect_column_values_to_be_unique("id")
expect_column_values_to_be_between(
    "price", min_value=0, max_value=10000)
expect_column_values_to_be_in_set(
    "status", ["active", "closed", "pending"])
expect_column_values_to_match_regex(
    "email", r"^[\w.]+@[\w.]+\.[a-z]{2,}$")

# Table-level expectations
expect_table_row_count_to_be_between(
    min_value=1000, max_value=10_000_000)
expect_table_columns_to_match_ordered_list(
    ["id", "name", "price", "quantity", "ts"])

# Multi-column expectations
expect_compound_columns_to_be_unique(
    ["date", "symbol"])
expect_column_pair_values_a_to_be_greater_than_b(
    "end_time", "start_time")
```

Checkpoint Configuration

YAML — checkpoint config

```yaml
# great_expectations/checkpoints/trades.yml
name: trades_checkpoint
config_version: 1.0
class_name: Checkpoint
run_name_template: "trades_%Y%m%d"
validations:
  - batch_request:
      datasource_name: warehouse
      data_asset_name: trades
    expectation_suite_name: trades_quality
action_list:
  - name: store_validation_result
    action:
      class_name: StoreValidationResultAction
  - name: update_data_docs
    action:
      class_name: UpdateDataDocsAction
```

GX + Pandas Quick Start

Python — quick validation

```python
import great_expectations as gx
import pandas as pd

# Quick validation without project setup
df = pd.read_parquet("data/trades.parquet")

context = gx.get_context()  # ephemeral context
ds = context.sources.add_pandas("mem")
asset = ds.add_dataframe_asset("trades")

batch = asset.build_batch_request(
    dataframe=df
)
validator = context.get_validator(
    batch_request=batch,
    expectation_suite_name="quick_check",
    create_expectation_suite_with_name="quick_check",
)

# Run expectations interactively
validator.expect_table_row_count_to_be_between(
    min_value=100, max_value=1_000_000)
validator.expect_column_values_to_not_be_null("symbol")
validator.expect_column_mean_to_be_between(
    "price", min_value=1, max_value=1000)

# Get results
results = validator.validate()
print(f"Success: {results.success}")
print(f"Failed: {results.statistics["unsuccessful_expectations"]}")
```

GX Project Structure

```
great_expectations/
+-- great_expectations.yml       # main config
+-- expectations/
|   +-- trades_quality.json      # expectation suite
|   +-- orders_quality.json
+-- checkpoints/
|   +-- trades_checkpoint.yml    # validation config
+-- plugins/
|   +-- custom_expectations/
|       +-- expect_valid_ticker.py
+-- uncommitted/
    +-- data_docs/               # generated HTML reports
        +-- local_site/
            +-- index.html
```

Data Quality Strategy by Layer

Data quality strategy

```
# Bronze (raw): minimal checks
# - Schema matches expected columns
# - Row count > 0
# - No completely empty columns

# Silver (cleaned): data validity
# - Primary keys are unique and not null
# - Values in expected ranges
# - Foreign keys exist in dimension tables
# - Timestamps are in valid ranges

# Gold (aggregated): business logic
# - Totals match across tables
# - No negative revenue/quantities
# - Row counts within expected bounds
# - Distributions haven't shifted dramatically
```

Glossary of Key Terms

ACID	Atomicity, Consistency, Isolation, Durability — transaction guarantees for databases
Backfill	Re-running a pipeline for historical date ranges to fill in missing data
CDC	Change Data Capture — tracking row-level changes in a database
CTE	Common Table Expression — named subquery using WITH clause
DAG	Directed Acyclic Graph — workflow of tasks with dependencies (no cycles)
Data Lake	Storage layer that holds raw data in any format (typically object storage)
Data Lakehouse	Architecture combining data lake flexibility with warehouse reliability
Delta Lake	Open-source storage layer adding ACID transactions to data lakes
ELT	Extract, Load, Transform — load raw data first, transform in the warehouse
ETL	Extract, Transform, Load — transform data before loading into the warehouse
Grain	The level of detail in a fact table (e.g., one row per transaction)
IaC	Infrastructure as Code — managing infra through version-controlled config files
Idempotent	Operation that produces the same result regardless of how many times it runs
Medallion	Architecture pattern with bronze (raw), silver (cleaned), gold (aggregated) layers
Partition	Physical division of data by a key (e.g., date) for faster queries
SCD	Slowly Changing Dimension — techniques for tracking dimension changes over time
Schema Registry	Service that stores and validates Avro/JSON schemas for Kafka messages
Surrogate Key	Artificial key (auto-increment or hash) used instead of natural business key
XCom	Airflow mechanism for passing small data between tasks (cross-communication)
Consumer Group	Set of Kafka consumers that share work reading from topic partitions
Offset	Kafka's pointer tracking which messages a consumer has processed
Bloom Filter	Probabilistic data structure for fast "element not present" checks
Compaction	Merging small files into larger ones for better read performance
Materialized View	Pre-computed query result stored as a table, refreshed on demand
Window Function	SQL function that operates on a set of rows related to the current row
Star Schema	Dimensional model with a central fact table joined to dimension tables
Fact Table	Table storing measurable events (transactions, clicks, trips) at defined grain
Dimension Table	Table storing descriptive context (dates, locations, customers)